# 7 Three Ways to Access a Base Class




## KEY IDEAS

| # | Form | Notes |
|---|---|---|
| 1 | `Base.method(self)` | By name; hard-wired, ignores the MRO |
| 2 | `super().method()` | **Preferred**; follows the MRO |
| 3 | `super(Child, self).method()` | Explicit form of 2 (older style) |

**TIP** Introspection: `Child.__bases__` (tuple of direct bases), `Child.__base__` (primary base), `Child.__mro__` (full order).

**PITFALL** Calling a base by name bypasses the MRO. In a diamond hierarchy the shared base can run **twice** or be skipped; use `super()` everywhere.

## EXAMPLE

```python
class Base:
    def hello(self):
        return "Base"

class A(Base):
    def hello(self):
        return Base.hello(self) + "+A"          # 1) by name

class B(Base):
    def hello(self):
        return super().hello() + "+B"           # 2) super()

class C(Base):
    def hello(self):
        return super(C, self).hello() + "+C"    # 3) explicit

print(A().hello(), B().hello(), C().hello())
# → Base+A Base+B Base+C
print(C.__bases__[0].__name__, C.__base__ is Base)
# → Base True
```